

Lexical Analysis: Problem Set

One exercise per method, one per lexing concept, and true/false questions, with solutions

How to use this set

Part A has one exercise per method of Part II of *Lexical Analysis: The Complete Guide* (Methods 1 to 7). Part B has one exercise per lexing concept of Parts I and III: tokens and positions, hand-written lexers, regular expressions for tokens, conflict rules, scanning with pushback, lexer states, flex, and deriving a token pattern from its DFA. Part C is a set of true/false statements. Each solution sits directly under its question.

All grammars and expressions here are new: none is the running example of the guide or a problem from Problem Sets 2 to 4. Every answer was checked by program or by running `flex`. Difficulty: ★ direct, ★★ several steps, ★★★ proof or design.

Part A: The automata methods

A1. Thompson's construction (Method 1) (★★)

Let $r = (a \mid bc)^* b$.

(a) Parse r and count its leaves, unions, stars and concatenations.

Solution

$r = ((a \mid b \cdot c)^*) \cdot b$. Leaves a, b, c, b : 4. Unions: 1. Stars: 1. Concatenations: 2 ($b \cdot c$, and the star with the final b).

(b) Build $N(r)$ with convention B (concatenation merges states). List the states and edges, and give the initial and final state.

Solution

- $N(a)$: $2 \xrightarrow{a} 3$. $N(bc)$: $4 \xrightarrow{b} 5 \xrightarrow{c} 6$ (state 5 is the merge point).
 - Union: $1 \xrightarrow{\varepsilon} 2$, $1 \xrightarrow{\varepsilon} 4$, $3 \xrightarrow{\varepsilon} 7$, $6 \xrightarrow{\varepsilon} 7$.
 - Star: $0 \xrightarrow{\varepsilon} 1$, $7 \xrightarrow{\varepsilon} 1$ (repeat), $7 \xrightarrow{\varepsilon} 8$, $0 \xrightarrow{\varepsilon} 8$ (skip).
 - Final concatenation with $N(b)$: its initial state is merged into 8, giving $8 \xrightarrow{b} 9$.
- Initial state 0, final state 9. States 0–9: **10 states, 12 edges**.

(c) Check the counts against the formula, check property (P3), and give the counts under convention A.

Solution

Convention B: $|Q| = 2(4 + 1 + 1) - 2 = 10$ and $|E| = 4 + 4 \cdot 2 = 12$. (P3): states 0, 1 and 7 have two outgoing ε -edges; 2, 3, 4, 5, 6 and 8 have exactly one edge; 9 is final. Convention A adds one state per concatenation back and one ε -edge each: $2 \cdot 6 = 12$ states and $12 + 2 = 14$ edges.

A2. Simulating an NFA (Method 2) (★)

The NFA N over $\{a, b\}$ has start state 0, accepting state 5, and edges

$$0 \xrightarrow{\varepsilon} 1, \quad 0 \xrightarrow{\varepsilon} 4, \quad 1 \xrightarrow{a} 2, \quad 2 \xrightarrow{b} 3, \quad 3 \xrightarrow{\varepsilon} 0, \quad 4 \xrightarrow{a} 5, \quad 4 \xrightarrow{b} 6, \quad 6 \xrightarrow{b} 5.$$

(a) Simulate N on **abbb**, **aba** and **abab**. Give the state set after each character and the verdict.

Solution

Start: $\varepsilon\text{-closure}(\{0\}) = \{0, 1, 4\}$.

- **abbb**: $\{0, 1, 4\} \xrightarrow{a} \{2, 5\} \xrightarrow{b} \{0, 1, 3, 4\} \xrightarrow{b} \{6\} \xrightarrow{b} \{5\}$: **accept**.
- **aba**: $\{0, 1, 4\} \xrightarrow{a} \{2, 5\} \xrightarrow{b} \{0, 1, 3, 4\} \xrightarrow{a} \{2, 5\}$: contains 5, **accept**.
- **abab**: $\dots \xrightarrow{a} \{2, 5\} \xrightarrow{b} \{0, 1, 3, 4\}$: no 5, **reject**.

After the second **b** of **abbb**, the move from $\{0, 1, 3, 4\}$ reaches only 6: the closure is needed only when an ε -edge leaves the moved-to states.

(b) Describe $L(N)$ with a regular expression.

Solution

The loop $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$ reads ab any number of times; the exit through 4 reads a or bb . So $L(N) = L((ab)^*(a \mid bb))$ (checked on all strings up to length 8).

A3. Subset construction (Method 3) (★★)

Use the NFA N of A2.

(a) Run the subset construction. Name the DFA states A, B, ... in order of discovery and give the transition table and the accepting states.

Solution

DFA	NFA set	on a	on b	Accept?
A	$\{0, 1, 4\}$	$\{2, 5\} = B$	$\{6\} = C$	
B	$\{2, 5\}$	dead	$\varepsilon\text{-closure}(\{3\}) = \{0, 1, 3, 4\} = D$	yes
C	$\{6\}$	dead	$\{5\} = E$	
D	$\{0, 1, 3, 4\}$	B	C	
E	$\{5\}$	dead	dead	yes

Five states plus the dead state.

(b) Two of these states are equivalent. Which, and why can you see it without the table-filling algorithm?

Solution

A and D: both are non-accepting and they have identical rows (on a to B, on b to C), so they accept exactly the same suffixes. The minimal DFA has 4 states plus the dead state.

A4. Minimisation by table filling (Method 4) (★★)

A DFA over $\{a, b\}$ with start A and accepting state E:

	A	B	C	D	E	F
a	B	B	D	A	D	D
b	C	C	C	E	F	C

(a) Fill the distinguishability table: Rule 1, then sweeps of Rule 2. For each marked pair give the sweep and a witness string.

Solution

- **Rule 1:** (A,E), (B,E), (C,E), (D,E), (E,F). Witness ε .
- **Sweep 1:** (A,D), (B,D), (C,D) and (D,F). On b each of them goes to the pair (C,E), marked by Rule 1. Witness b .
- **Sweep 2:** (A,C), (A,F), (B,C), (B,F), all on a , which leads to (B,D), marked in sweep 1. Witness ab .
- **Sweep 3:** nothing new. Stop. Unmarked: (A,B) and (C,F).

(b) Give the minimal DFA and describe its language.

Solution

Merge $P = \{A, B\}$ and $Q = \{C, F\}$, relabelling every arrow into B as one into P and into F as one into Q :

$$P \xrightarrow{a} P, P \xrightarrow{b} Q; \quad Q \xrightarrow{a} D, Q \xrightarrow{b} Q; \quad D \xrightarrow{a} P, D \xrightarrow{b} E; \quad E \xrightarrow{a} D, E \xrightarrow{b} Q.$$

Four states, start P , accepting E . The states remember the longest suffix that is a prefix of bab : nothing, b , ba , bab . So the language is $(a \mid b)^*bab$, the strings ending in bab (checked up to length 9).

A5. State elimination, DFA to regex (Method 5) (★★)

The minimal DFA from A3: states P (start), B , C , E ; accepting B and E ; edges $P \xrightarrow{a} B$, $P \xrightarrow{b} C$, $B \xrightarrow{b} P$, $C \xrightarrow{b} E$ (all other moves go to the dead state).

(a) Add a final state f and eliminate C , then E , then B . Give the resulting regular expression.

Solution

Add $B \xrightarrow{\varepsilon} f$ and $E \xrightarrow{\varepsilon} f$.

- Eliminate C (no loop): the path $P \xrightarrow{b} C \xrightarrow{b} E$ becomes $P \xrightarrow{bb} E$.
- Eliminate E : $P \xrightarrow{bb} E \xrightarrow{\varepsilon} f$ becomes $P \xrightarrow{bb} f$.
- Eliminate B (no loop): it is entered from P on a and leaves to P on b and to f on ε . New edges: $P \xrightarrow{ab} P$ (a loop on P) and $P \xrightarrow{a} f$, which joins $P \xrightarrow{bb} f$ to give $P \xrightarrow{a|bb} f$.

With loop ab on the start and edge $a \mid bb$ to f : $(ab)^*(a \mid bb)$, the language of A2, as it must be.

A6. Brzowski derivatives (Method 6) (★★)

Let $R = (ab \mid b)^*$.

(a) Compute $\partial_a R$ and $\partial_b R$, then the derivatives of every new expression, until no new expression appears. Mark which are nullable.

Solution

$\partial_a R = \partial_a(ab \mid b) R = (b \mid \emptyset) R \equiv bR$. $\partial_b R = \partial_b(ab \mid b) R = (\emptyset \mid \varepsilon) R \equiv R$.

$\partial_a(bR) = (\partial_a b) R \mid \nu(b) \partial_a R = \emptyset R \mid \emptyset \equiv \emptyset$. $\partial_b(bR) = \varepsilon R \mid \emptyset \equiv R$.

Three expressions: R (nullable, since a star accepts ε), bR (not nullable), and $\emptyset$ (dead).

(b) Draw the resulting DFA, test **bab** and **aab**, and describe the language in words.

Solution

$R \xrightarrow{a} bR$, $R \xrightarrow{b} R$, $bR \xrightarrow{b} R$, $bR \xrightarrow{a} \emptyset$; accepting: R . **bab**: $R \rightarrow R \rightarrow bR \rightarrow R$, nullable: **accept**. **aab**: $R \rightarrow bR \rightarrow \emptyset$: **reject**. The language is the strings over $\{a, b\}$ in which every a is immediately followed by a b (checked up to length 10).

A7. Lower bounds and non-regularity (Method 7) (★★★)

(a) Let L_4 be the strings over $\{a, b\}$ whose number of a 's is divisible by 4. Prove that every DFA for L_4 has at least 4 states, and give a DFA with exactly 4.

Solution

Take ε, a, aa, aaa . For $0 \leq i < j \leq 3$, append a^{4-i} : $a^i a^{4-i} = a^4 \in L_4$, but $a^j a^{4-i}$ has $4 + (j - i)$ a 's with $0 < j - i < 4$, so it is not in L_4 . The four strings are pairwise distinguishable, so any DFA needs 4 states. A DFA with states $0, 1, 2, 3$ (the number of a 's modulo 4), start and accept 0 , $q \xrightarrow{a} q + 1 \bmod 4$, $q \xrightarrow{b} q$, has exactly 4. So the minimal DFA has 4 states. ■

(b) Prove that $L = \{a^n b^n : n \geq 0\}$ is not regular.

Solution

The infinitely many strings $a^0, a^1, a^2, \dots$ are pairwise distinguishable: for $i \neq j$ the suffix b^i gives $a^i b^i \in L$ and $a^j b^i \notin L$. A DFA would need a different state for each of them, hence infinitely many states. So no DFA, and no regular expression, describes L . ■

(c) Explain in two sentences what (b) says about comments that may be nested.

Solution

Read a as $/*$ and b as $*/$: properly nested comments contain the shape $a^n b^n$, so a lexer must count the depth, which no regular expression can do. That is why nested comments need a depth counter in a lexer state (exercise B6).

Part B: Lexing concepts

B1. Tokens, attributes and positions (★)

A lexer for a C-like language skips blanks and `//` comments. Integers are `[0-9]+`, floats follow decimal `("e"|"E") ("+"|"-"?) digit+ or decimal`, identifiers are `[a-z][a-z0-9]*`, and `if` is a keyword. The input is (columns start at 1):

```
1 if (x>=10) y = x * 2.5e-1;  
2   s = "hi\n"; // done
```

(a) List every token with its class, its attribute (if any), and its line:column.

Solution

Lexeme	Class	Attribute	Pos.	Lexeme	Class	Attribute	Pos.
if	keyword	—	1:1	x	identifier	x	1:16
(	punctuation	—	1:4	*	operator	—	1:18
x	identifier	x	1:5	2.5e-1	float	0.25	1:20
>=	operator	—	1:6	;	punctuation	—	1:26
10	integer	10	1:8	s	identifier	s	2:3
)	punctuation	—	1:10	=	operator	—	2:5
y	identifier	y	1:12	"hi\n"	string	hi + newline	2:7
=	operator	—	1:14	;	punctuation	—	2:13

(b) How many tokens are there? What is the difference between the string token's lexeme and its attribute? Why is `>=` one token?

Solution

16 tokens; the comment and the blanks produce none. The lexeme `"hi\n"` is 6 source characters; the attribute is the interpreted 3-character string `h`, `i`, newline. `>=` is one token by longest match: the rule for `>=` matches 2 characters, the rule for `>` only 1.

B2. A hand-written lexer bug (★)

```
int n = 0, c;  
while ((c = input.read()) >= '0' && c <= '9')  
    n = n * 10 + (c - '0');  
return new Token(INT, n);
```

(a) Trace this code on `12+3`. Which tokens does the lexer produce? Why does `12 + 3` seem to work?

Solution

It reads `1`, `2` (so `n = 12`), then reads `+`, which fails the test, and returns `INT(12)`. The `+` has been consumed and is lost; the next call returns `INT(3)`. Output: `INT(12) INT(3)`, with the operator missing. With `12 + 3`, the consumed character is a blank, which would have been skipped anyway, so the bug is invisible.

(b) Fix it.

Solution

Peek before consuming: `while (Character.isDigit(input.peek())) n = n * 10 + (input.read() - '0');`. Alternatively, read one character too many and push it back (`ungetc` in C).

B3. Writing token patterns (★★)

(a) Write a regular expression for binary literals: `0b` or `0B`, then binary digits, where single underscores may appear *between* digits (as in `0b1010_0101`). Give four strings it must reject.

Solution

`0[bB][01](_[01])*`. It accepts `0b1`, `0b10_01`, `0B1_0_1`. It rejects `0b` (no digit), `0b_1` (leading underscore), `0b1_` (trailing underscore), `0b1__0` (double underscore) and `0b12` (not binary). Each underscore is forced to be followed by a digit by the group `(_[01])`.

(b) Write a regular expression for 24-hour times `HH:MM` (`00:00` to `23:59`). Test it on `19:07`, `24:00`, `7:30` and `12:60`.

Solution

`([01][0-9]|2[0-3]):[0-5][0-9]`. `19:07` is accepted; `24:00` is rejected (hour 24), `7:30` too (one-digit hour) and `12:60` too (minute 60). The hour is split into ranges because `[0-2][0-9]` would accept 29.

B4. Longest match and rule order (★★)

A flex specification lists, in this order: `"=="` EQ, `"="` ASSIGN, `">"` ARROW, `[a-z]+` ID, `"else"` ELSE, `[0-9]+` INT, `[0-9]+."` `[0-9]+` FLOAT, `."` DOT, and a rule skipping blanks.

(a) Give the token stream for `else elsewhere=>x==3.14.5`.

Solution

ID(`else`) ID(`elsewhere`) ARROW ID(`x`) EQ FLOAT(`3.14`) DOT INT(`5`) (output of flex).

- `else`: ID and ELSE both match 4 characters; ID is written first, so it wins.
- `=>`: 2 characters beat ASSIGN's 1. `==` likewise beats `=`.
- `3.14.5`: FLOAT matches `3.14` (4 characters), INT only 3. Then `.` is DOT and `5` is INT.

(b) What does flex report about this specification, and how do you fix it?

Solution

"rule cannot be matched" for the ELSE rule: whenever it matches, ID matches the same characters and comes first. Move `"else"` above `[a-z]+`. Longest match still keeps `elsewhere` an identifier.

B5. Scanning with pushback (★★)

Rules, in order: T1 `ab`, T2 `abcd`, T3 `c`, T4 `b`. A read counts every character fetched, including the one on which the scanner gets stuck; reaching the end of the input is not a read.

(a) Draw the combined DFA with its token labels.

Solution

States: start 0; $0 \xrightarrow{a} 1$ (no label), $0 \xrightarrow{b} 2$ (T4), $0 \xrightarrow{c} 3$ (T3), $1 \xrightarrow{b} 4$ (T1), $4 \xrightarrow{c} 5$ (no label), $5 \xrightarrow{d} 6$ (T2). All other moves go to the dead state.

(b) Scan `abcbcd`. Give each token, the number of reads it costs, and its pushback. Give the total number of reads.

Solution

- From position 0: `a` (state 1), `b` (state 4, T1), `c` (state 5), `a` stuck. Emit **T1 ab**; 4 reads; push back `c`, `a` (2).
 - `c` (state 3, T3), then `a` stuck. Emit **T3 c**; 2 reads; push back 1.
 - `a`, `b`, `c`, `d` (state 6, T2), end of input. Emit **T2 abcd**; 4 reads; no pushback.
- Total: 10 reads for 7 characters.

(c) What is the maximum pushback of this specification, over all inputs?

Solution

After an accepting state, the longest run of non-accepting states is one state (5, reached from 4 by `c`). So the maximum pushback is $1 + 1 = 2$, reached by `abc` followed by anything but `d`.

B6. Lexer states: indentation and nested comments (★★)

(a) Give the `INDENT` and `DEDENT` tokens, with the line that triggers each, for this Python file. Then say what happens if line 3 of a second file is indented by 2 spaces after a line indented by 4 (lines: 0, 4, 2 spaces).

```
1 if x:
2     y = 1
3     if y:
4         z = 2
5
6     w = 3
7 v = 4
```

Solution

Stack [0]. Line 2 (4 spaces): push 4, **INDENT**. Line 3 (4): nothing. Line 4 (8): push 8, **INDENT**. Line 5 is blank and ignored. Line 6 (4): pop 8, **DEDENT**. Line 7 (0): pop 4, **DEDENT**. Python's tokenizer gives exactly `INDENT@2`, `INDENT@4`, `DEDENT@6`, `DEDENT@7`.
Second file: the stack is [0, 4] at line 3; $2 < 4$, so pop 4 (**DEDENT**); now the top is $0 < 2$ but $0 \neq 2$: **inconsistent dedent**. Python reports "IndentationError: unindent does not match any outer indentation level" at line 3.

(b) A lexer handles comments with a depth counter: `/*` increments it (entering `COMMENT` mode at depth 1), `*/` decrements it, and the lexer leaves `COMMENT` mode at depth 0. Trace the depth on `/* a /* b */ c */ d */`. Which tokens follow? What would a two-state `COMMENT` lexer without a counter produce?

Solution

Depth: 1 at the first `/*`, 2 at the second, 1 after the first `*/`, 0 after the second `*/`: back to code. Then `d` is an identifier and the final `*/` is lexed as code: two operator characters (or an error). Without a counter, the comment ends at the *first* `*/`, and `c */ d */` is lexed as code.

B7. Predicting flex output (★★)

```
%%
x+y?    printf("<1:%s>", yytext);
xy       printf("<2>");
y+       printf("<3:%d>", (int) yyleng);
x*z      printf("<4:%s>", yytext);
.|\n    printf("[%s]", yytext[0] == '\n' ? "NL" : yytext);
```

(a) Predict the output on `xyyyzxxz`, a newline, then `yxw`. Is rule 2 ever used?

Solution

`<1:xy><3:2><4:z><4:xx>[NL]<3:1><1:x>[w]` (output of `flex`).

- `xy`: rule 1 matches 3 characters; rule 4 cannot (no `z` after `xx`).
- `yy`: rule 3, length 2.
- `z`: rule 4 with zero `x`'s.
- `zx`: rule 1 matches `x` (1), rule 4 matches `zx` (2), so rule 4 wins.
- After the newline: `y` (rule 3, length 1), `x` (rule 1), `w` (fallback).

Rule 2 is never used: on `xy`, rule 1 matches the same 2 characters and is written first. `flex` warns “rule cannot be matched”.

B8. A token pattern from its DFA (★★★)

In SQL and Pascal, a string is written in single quotes, and a quote inside the string is written twice: `'it's'` is the string `it's`. Strings cannot contain a raw newline.

(a) Draw the DFA over the classes `'`, newline and *other*, then read off a regular expression.

Solution

$0 \xrightarrow{' } 1$; $1 \xrightarrow{\text{other}} 1$; $1 \xrightarrow{' } 2$ (accepting: a complete string); $2 \xrightarrow{' } 1$ (the doubled quote, back inside). Everything else goes to the dead state. State 1 means “inside the string”; state 2 means “just saw a quote that may close the string”.

Inside, the machine either stays in 1 on `[^'\n]` or makes the round trip $1 \rightarrow 2 \rightarrow 1$ on `'`. It leaves through one `'`. Hence `'([^\n]|')*`.

(b) With a rule for blanks and a one-character error rule, what does `flex` produce on `'it's'` `'x'`, on `''''` and on `'ab'`? Explain the last one.

Solution

`STR('it's')` `STR('x')`; `STR('')'` `STR('')`; and `STR('ab')` `ERR(')` (output of `flex`). On `'ab'`, the DFA accepts after `'ab'` (state 2), reads the next `'` back into state 1 (an escaped quote, so still inside), and reaches the end of input without closing. Longest match falls back to the last accepting position: `'ab'`, and the leftover `'` is an unterminated string, reported as an error.

Part C: True or false

Decide, and justify in one line.

- C1. A DFA may have ε -transitions.
- C2. Every NFA with n states has an equivalent DFA with at most 2^n states.
- C3. Under convention A, Thompson's NFA has at most $2 \times (\#operators + \#operands)$ states.
- C4. Minimising a DFA can change the language it accepts.
- C5. Two DFAs accept the same language if and only if their minimal DFAs are identical up to renaming.
- C6. Two states with identical rows, both accepting or both non-accepting, are equivalent.
- C7. $\{a^n b^n : n \geq 0\}$ is a regular language.
- C8. Longest match always chooses the rule written first.
- C9. Rule order matters only when two rules match the same longest lexeme.
- C10. In flex, the pattern `.` matches a newline.
- C11. A catch-all error rule should be written first in the specification.
- C12. ε -closure(S) always contains S .
- C13. Simulating a Thompson NFA on an input of length n costs $O(n|Q|)$.
- C14. The scan-and-backtrack algorithm may read the same character more than once.
- C15. Nested comments can be described by one regular expression.
- C16. Abbreviations in a lexer specification may be recursive.
- C17. The attribute of a string token is its raw source text, quotes included.
- C18. Python's INDENT and DEDENT tokens can be produced by a finite automaton alone.
- C19. A regular language has a unique minimal DFA, but many regular expressions describe it.
- C20. Hopcroft's algorithm and table filling can give different minimal DFAs for the same input.

Answers

- C1 False: ε -moves make it an NFA.
- C2 True: each DFA state is a set of NFA states.
- C3 True: exactly $2(\text{leaves} + \#| + \#*)$, and concatenations are operators too.
- C4 False: it only merges states that accept the same suffixes.
- C5 True: the minimal DFA is unique (Myhill–Nerode).
- C6 True: they go to the same states on every symbol, so they accept the same suffixes.
- C7 False: infinitely many pairwise distinguishable prefixes a^i (A7).
- C8 False: length first; order only breaks ties.
- C9 True.
- C10 False: `.` excludes newline; use `.\n`.
- C11 False: it must be last, or it steals one-character tokens.
- C12 True: zero ε -steps are allowed.
- C13 True: each state has at most two outgoing edges (P3).
- C14 True: pushed-back characters are read again (B5).
- C15 False: nesting needs a counter (A7c).
- C16 False: that would give context-free power.
- C17 False: it is the interpreted value (B1).
- C18 False: it needs a stack of widths.
- C19 True: e.g. $(a|b)^*abb$ and $(b|a(a|ba|bba)^*bbb)^*a(a|ba|bba)^*bb$.
- C20 False: both give the unique minimal DFA, up to renaming.